

RO034

Using Machine Learning and Binary Visualisation to identify Malicious URLs

Done by:

Chia Sin Jie 22S6D

Liong Kai Rui Reyes (L) 22S6D

Owen Sebastian Saputra 22S6D

Mentors:

Ms Yan Youyu

Dr Fedor Duzhin

Abstract: To minimise the chances of people falling into the traps of an increasing number of malicious websites, we aim to create an accurate model with low access time to differentiate benign, defacement and phishing websites. To achieve this, the HTML code from these websites were scraped and stored into a binary file, binary visualisation was performed using BinVis to create 2D images from the binary files. Finally, the machine learning model MobileNet was used to create the model which achieved a high overall average sparse categorical accuracy of 80%.

1. Introduction

As technology evolves, the dangers of malicious URLs that infect users with various malware to steal and destroy data on a large scale has only been increasing.¹ In the US alone, the total damage caused by malicious URLs have amounted to more than \$7 billion in 2021.² Thus, it is critical to identify and remove them quickly and easily to reduce the risk to netizens.

Over time, many measures like antivirus softwares have been created that make use of heuristic-based detection and blacklist-based methods to check for malicious websites. However, their effectiveness is limited as they operate on the basis of experience, and are likely to overlook novel variants of malicious URLs. Additionally, as malicious actors are constantly finding ways to bypass existing methods, the effectiveness of heuristic-based detection is further undermined.

To overcome the aforementioned constraints, we propose a computer vision-based approach that makes use of binary visualisation. This method's main advantage is in its improved ability to automatically detect unknown malicious URLs, through the use of deep learning. We were inspired by projects utilising binary visualisation to discern malware files with high accuracy³. Our project aims to extend the use of this method to detect various types of malicious websites, including phishing and defacement websites. Our group focused on websites as it has the most potential to protect users from scams⁴.

¹ Jr, Ron Samson. "What Are Malicious Urls and Links? How to Identify and Fight Them." ClearNetwork, Inc, December 7, 2021.
<https://www.clearnetwork.com/malicious-urls/#:~:text=A%20malicious%20URL%20is%20one,ly%20or%20a%20modified%20hyperlink>.

² Chang, Jenny. "119 Impressive Cybersecurity Statistics: 2021/2022 Data & Market Analysis." Financesonline.com. FinancesOnline.com, January 14, 2022.
[https://financesonline.com/cybersecurity-statistics/#:~:text=Cybercrime%20Statistics,-One%20of%20the&text=Global%20cybercrime%20damage%20in%202021,\(Cybercrime%20Magazine%2C%202020\)](https://financesonline.com/cybersecurity-statistics/#:~:text=Cybercrime%20Statistics,-One%20of%20the&text=Global%20cybercrime%20damage%20in%202021,(Cybercrime%20Magazine%2C%202020)).

³ *Binary visualisation for malware detection - university of Plymouth*. (n.d.). Retrieved December 25, 2022, from

https://pearl.plymouth.ac.uk/bitstream/handle/10026.1/14179/TPSS-2018-Vol11n1_223-237Baptista.pdf

⁴ Swns. (2019, December 6). *More than 60% of Americans say they've been a victim of an online scam*. New York Post. Retrieved December 25, 2022, from
<https://nypost.com/2019/12/06/more-than-60-of-americans-say-theyve-been-a-victim-of-an-online-scam/>

2. Hypothesis Of Research

By using binary visualisation to represent websites' source code in a 2D image, the inherent differences between the content in malicious and legitimate URLs can be visually identified, thus allowing computer-vision models to quickly and accurately discern between them.

3. Methodology

To distinguish between legitimate and malicious URLs, we propose a method with 2 stages:

1. Visualisation
 - a. The source code of the URL in question is scraped and stored in a text file
 - b. The text files are converted into 2D images through binary visualisation
2. Detection
 - a. The images are analysed by a trained convolutional neural network

A: Visualisation stage

Firstly, data from a variety of datasets were collected, consisting of Benign, Phishing, and Defacement URLs. In total, 1546 URLs from various sources were collected (Figure 1).

Classification	Number of samples	Source
Benign	709	Kaggle, CIC-Bell ⁵
Phishing	723	Phishtank ⁶
Defacement	395	Kaggle ⁷

Figure 1: Number of samples per classification type

⁵Siddhartha, M. (2021, July 23). *Malicious urls dataset*. Kaggle. Retrieved July 31, 2022, from <https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>

⁶ [PhishTank | Join the fight against phishing](#)

⁷Siddhartha, M. (2021, July 23). *Malicious urls dataset*. Kaggle. Retrieved July 31, 2022, from <https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>

The source code from the above URLs are scraped using the HTTP requests library and stored in a text file, preventing users from accessing potentially dangerous content on malicious websites (Figure 2). To ensure that only active websites were scraped, data was only collected from websites that sent a valid HTTP 200 response code.

```
import requests

f = open('benign.csv', 'r') #file containing valid urls
line = f.readline()
x = 0

while line != '':
    URL = line

    try:
        print(URL)
        response = requests.get(URL)
        if response.status_code == 200:
            html = response.get(URL).text

            with open("benign%d.txt" % x, "w") as test_file:
                page = bytes(html, 'utf-8')
                test_file.write(str(page))
                x += 1
            except:
                print("error")

        line = f.readline()

f.close()
```

Figure 2: URL scraping script

Binary visualisation is then performed using BinVis, a public library which converts binary files into 2D images by employing colour schemes to represent various bytes⁸ (Figure 3). BinVis uses a clustering algorithm based on the Hilbert space-filling curve⁹. It maps binary files, such as our text files, onto a 2D image while keeping elements that are close together as near as possible for locality preservation. The detail colour scheme is used, which assigns a colour to each different byte value. To maximise the difference between colours yet ensure similar elements have similar colours, the 3D Hilbert curve traversal of the RGB colour cubed is hence used.¹⁰ The final output of Binvis is an image with a pre-set size of 256 x 1028 pixels.



Figure 3: Detail Class colour scheme

⁸ "Binvis.io." binvis.io. Accessed July 28, 2022. <https://binvis.io/>.

⁹ "CMU School of Computer Science." Accessed July 28, 2022. <https://www.cs.cmu.edu/~christos/PUBLICATIONS/ieee-tkde-hilbert.pdf>.

¹⁰ <http://binvis.io/#/view/examples/metricprops.pdf.bin?modal=help>

Lastly, the images produced were resized to 224 x 224 pixels, the optimal image size for the MobileNet model used in the detection stage.

B: Detection Stage

After obtaining the preprocessed images, they were split into training (70%), validation (20%), and test datasets (10%), each with the following purposes:

Training dataset: Data used to fit the model

Validation dataset: Data used to evaluate the model's generalisation ability during training

Testing dataset: Data used to provide an unbiased evaluation of a final trained model

To detect malicious URLs, a convolutional neural network was constructed on Tensorflow with the base model MobileNet V2. MobileNet was selected as it is a lightweight deep convolutional neural network network that has low latency for mobile and embedded devices. Its excellent image recognition ability and fast detection rates minimises time and memory required for malicious URLs to be classified, thus making predictions efficiently.¹¹

To train the model, the pretrained model's convolutional base was first frozen, and a classification head consisting of a GlobalAveragePooling¹² layer and a Dense layer was added. Our model was fitted with a learning rate of 0.001, Adam optimizer, softmax activation function¹³ and the Sparse Categorical Cross-Entropy loss function.

Our model was trained for a total of 40 epochs with a batch size of 32 on a virtual machine, running on Intel Xeon CPU @2.20 GHz, with 13 GB RAM, NVIDIA Tesla K80 accelerator, and 12 GB GDDR5 VRAM. Lastly, our model was fine-tuned by unfreezing the base model and setting the top 10 layers of the base model to be trainable.

¹¹ Howard, Andrew G., Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." arXiv.org, April 17, 2017. <https://arxiv.org/abs/1704.04861>.

<https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>

¹² https://www.tensorflow.org/api_docs/python/tf/keras/layers/GlobalAveragePooling2D

¹³ <https://paperswithcode.com/method/softmax>

4. Results and Discussion

Our model's overall performance was assessed using the Sparse Categorical Accuracy metric¹⁴. The metric calculates how often our model's predictions match the images' integer labels, based on the formula shown below.

$$Accuracy = \frac{True_{positive} + True_{negative}}{True_{positive} + True_{negative} + False_{positive} + False_{negative}}$$

Where,

True positive = Number of images correctly classified as malicious URLs

True negative = Number of images correctly classified as legitimate URLs

False positive = Number of images incorrectly classified as malicious URLs

False negative = Number of images incorrectly classified as legitimate URLs

The model took a total of 160 seconds to complete 40 training epochs. The training and validation sparse categorical accuracy rates are high (90% and 81% respectively), while the training and validation loss values are low (30% and 55% respectively). (Figure 4) Our model also performed similarly well on the test set (80% accuracy), suggesting that the model successfully learned the input and could effectively classify the various URL types.

Train		Validation		Test	
Loss	Accuracy	Loss	Accuracy	Loss	Accuracy
0.3085	0.9003	0.5505	0.8185	0.4861	0.8000

Figure 4: Loss and Accuracy rates

The training and validation loss curves decreased to a point of stability with a small gap between the 2 final loss values, suggesting a well-trained model with minimal overfitting (Figure 5).

¹⁴ https://www.tensorflow.org/api_docs/python/tf/keras/metrics/SparseCategoricalAccuracy



Figure 5: Learning curve for Training and Validation Loss

The confusion matrix for our model's predictions on the test set is shown in Figure 6. It shows how often each image is predicted correctly and incorrectly due to confusion between the 3 classes.¹⁵

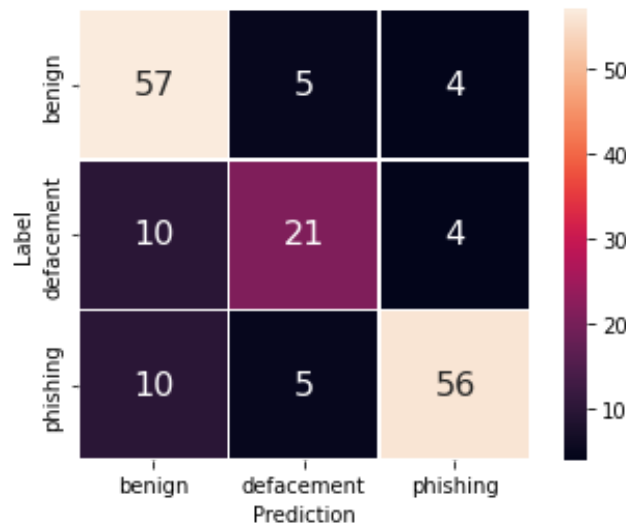


Figure 6: Confusion Matrix

As the diagonal values which represent the number of images correctly classified by the model are higher, it indicates good model performance in general, although the model tends to be less accurate in identifying defacement websites. A possible explanation for this is that the alterations on defacement websites are often subtle¹⁶ and thus difficult for the model to detect.

¹⁵ <https://towardsdatascience.com/understanding-confusion-matrix-a9ad42dcfd62>

¹⁶ <https://www.cisecurity.org/insights/spotlight/cybersecurity-spotlight-website-defacements>

As for differentiating benign and malicious websites, the 2D images produced during the binary visualisation stage reveal unique patterns for the different classes. Legitimate websites tend to have a more detailed RGB value compared to malicious websites. This may be due to additional characters from licences, hyperlinks, and detailed data entry forms (Figure 7). Meanwhile, malicious websites generally contain little CSS reference, multiple images and a single login form with no security scripts, creating a smaller data input string when scraped.

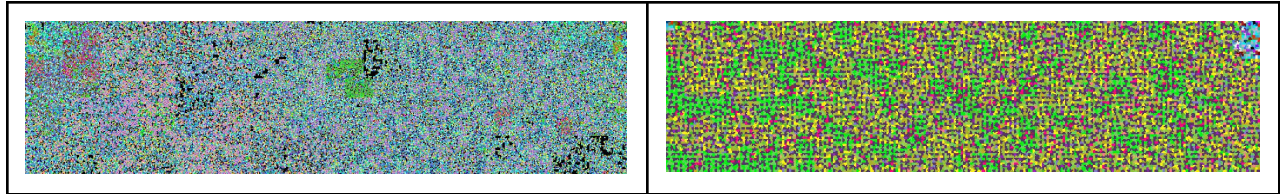


Figure 7: Example of malicious vs non-malicious URL

5. Conclusion

Our research has shown that using binary visualisation and the pretrained model MobileNet V2 is a viable method for detecting malicious URLs. While this approach is less effective at detecting certain types of malicious URL, such as defacement sites, our model is fairly accurate, achieving a sparse categorical accuracy rate of 80% on test data. This is due to the visual representation techniques used, which offer great insight into the inherent differences in code structure between malicious and non-malicious URLs. Furthermore, as the model is lightweight, it can be deployed onto mobile phones and embedded devices to perform detections with low access times, increasing accessibility.

In the future, we hope to deploy our model onto mobile platforms as a cloud-based cybersecurity software that performs real-time detection on URLs. Our model's accuracy rates also have significant room for improvement, such as using larger datasets and enhancing image-based feature extraction.

Bibliography